Checking or Recomputing? Two Cost Regimes in Verifying AI-Generated Formal Artifacts

Anonymous research draft

Abstract

AI-generated mathematics changes what proof verification asks the machine to do. A mathematical witness can be checked without repeating its producer's search, while a computational certificate can require the kernel to recompute the calculation encoded in the proof. We characterize these two cost regimes using released `openai/math` artifacts and their version-matched Mathlib dependency. The same comment/string-masked scanner finds 1,354,379 `decide +kernel` sites in 163,097 generated-library files, versus 7 sites in 8,529 Mathlib files: approximately 10,668.3 times the source-megabyte density. Only 5,848 generated-library files (3.5856%) contain this form. This is lexical concentration, not a measured fraction of verification time. Runtime-weighted prevalence remains [**RESULT: pending from the incomplete native sample**]. We examine repeated imports and production/checking coupling, and test single-process streaming and automatically chunked original-kernel checking as design-implication prototypes. [**RESULT: Matched independent-checking and whole-path parallel results.**] The meaning of *verified* is a separate boundary: the default build omits 33.30% of kernel-decision sites, outcomes depend on settings and observation, and whole-library axiom closure remains unestablished. The contribution is a characterization and its tested implications, not a general accelerator or a replacement trusted kernel.

1 Introduction

Checking a proof need not resemble producing it. Mathematical proof production can involve search, tactics and elaboration; an independent kernel can instead check the supplied witness. A computational proof exposes a different boundary: when its justification depends on reducing a large decidable expression, checking executes the relevant computation. Replaying the unchanged certificate can repeat that work.

Our primary workload is the released `openai/math` research library [20]. Its pinned Mathlib dependency supplies a version-matched reference [7]. The generated release contains far more source-level kernel evaluation, concentrated in a small subset of files. This motivates a distinction between mathematical-witness checking and proof-embedded recomputation, rather than treating verification as one uniform solver call. Files can mix both regimes; tokens cannot assign CPU.

The distinction suggests two systems responses. For mathematical proofs, can checking avoid producer elaboration and repeated import preparation? For computation, can independent claims be checked by several unchanged kernels and assembled into the original theorem? A faster untrusted numeric answer, weaker import trust scope or new axiom is not equivalent verification. We organize the study around four questions:

RQ1 Where do verification costs fall, and how prevalent are the computation-replaying and mathematical-proof-checking regimes across the library?

RQ2 Why does independent verification fail to realize the potential low cost of mathematical-proof checking?

RQ3 Can computational checking scale across cores or a cluster without expanding the trusted base?

RQ4 What gaps remain between a reported verified result and build coverage, reproducible outcomes and known axiom dependencies?

Each RQ separates evidence, missing results and design implications. We use public generated-software outputs and retained candidate streams as contrasts, never generate replacements for unavailable releases. Streaming and chunking are prototypes testing the characterization; warm imports, ordinary scheduling and external search/checking are prior techniques. Missing native profiles and full-library weights remain explicit.

2 Methodology

2.1 Sources and the common lexical boundary

The primary release is `4eaee87548250b2682affda7d05d947c05554bae`, with Lean 4.34.1 and Mathlib `d13f23b723b8a846827a245b89c10fc7d3f11612`. We compare all released Lean files in the primary inventory with the reference repository's `Mathlib/` subtree, excluding tests, archive and tooling directories from

that reference denominator. The reference source tree is checked at its exact Git revision. All 163,097 primary source members are independently rescanned from the pinned official archive and matched by path/byte size to the complete inventory; decoding and lexical checks succeed. The local extracted cache is partial and is not used as the population. A complete received-archive-prefix scan and full-archive scan agree exactly; they are repeated validation, not additional corpus observations. The same existing scanner erases nested comments and string literals and matches `decide`, whitespace, `+`, whitespace and `kernel`, allowing zero whitespace. One MB is 1,000,000 original UTF-8 source bytes. Sites and containing files are not dynamic calls, distinct propositions, accepted proofs or native time.

Vericoding releases 1,503 model-tagged Lean snapshots, 1,490 distinct source texts; its original 626 are a subset [30]. Publisher labels are not our native verdicts or axiom audit. The inspected Vero release does not establish availability of evaluated agent outputs [32]; scaffolds/references do not replace them. Lean Kernel Verify supplies checking code and KIR fixtures with separate output provenance [19]. **[RESULT: Version-qualified generated-software native costs and axiom audit.]** The contrast streams retain 7,808 miniF2F, 5,952 ProofNet per recorded pass and 3,566 AutoVerus sources, including unsuccessful attempts. Source-sorted replay is not historical arrival traffic.

2.2 Cost boundaries and regime classification

Source production includes imports, elaboration, tactics, proof construction, first checking and requested artifact emission. AI inference time is excluded unless separately supplied by publisher logs. Independent checking consumes compiled declarations under a declared trust scope. A kernel-only subclock omits preparation/cleanup and cannot replace either complete process clock.

The regimes describe checking work, not disjoint proof languages. Computational checking normalizes embedded calculations; mathematical checking can avoid the producer's search and elaboration. A module may contain both. A C++ checker sample includes inference, equality, normalization and cleanup, not solely final validation. User-cycle percentages are not elapsed-time shares. **[RESULT: No-memory-option per-declaration/module profiles, classification, observer uncertainty and runtime-weighted prevalence.]**

2.3 Sampling and current native point

The original frame remains 163,097 modules, with 21 nonempty strata formed from four source-size bins, three internal-import-depth bins and masked kernel-decision presence. Seed 20261030 selects two draws per stratum, 42 total. Inclusion probability is $2/N_h$; purposeful modules and prerequisites receive no additional population weight. For complete strata,

$$\widehat{T}_h = N_h \bar{y}_h, \qquad \widehat{V}_h = N_h^2 (1 - n_h/N_h) s_h^2 / n_h, \quad n_h = 2.$$

Missing draws stay missing, not replaced by completed-response counts. Two-draw normal-cost intervals are fragile under heavy tails. The newly checked no-`-M` point has 31 successful native calls: 29 original random draws and two purposeful modules. All original sources, no-`-M` arguments, PID/birth retirement and matching native clock sums are checked. The random-frame coverage is 29/42 with 12 complete strata; 13 draws remain missing. The two purposeful modules receive no sampling weight. These are local compilation costs with prerequisite objects available, not cold transitive builds, whole Lake execution or generation time. **[RESULT: Complete no-memory-option sample and per-regime cost estimates.]**

The platform is phobos-vt, Intel Core i9-14900K, observed 64,205,640 KiB RAM. Timed native captures share one P0–7 lock; preparation/analysis use E cores. Eight requested workers are not assumed to use eight CPU cores. All current commands omit `-M`, including `-M0`; memory protection is external cgroup. The W point uses 15 GiB charged native memory and separate one-second sampled native/controller RSS response. Charged pages, leader RSS and tree RSS differ; sampled response can overshoot and is not a kernel hard aggregate-RSS quota. Every result must record its actual enforcement boundary.

A measurement correction, not a workload finding. The pinned runtime initializes its native memory limit to zero; `check_memory` returns before RSS queries while zero [13]. Our earlier optional `-M` setting introduced periodic accounting. Three ordered plain repetitions per arm give guarded/no-`-M` wall-median ratios 2.03/1.61/1.01 for Grid/UMoments9/EnvelopePsiB, with matching verdicts, diagnostics and emitted artifacts. The same outer 15 GiB cgroup remains; output and heartbeat settings match within each module. These passing cases are not resource-failure equivalence. The earlier five-source `openai/math` panel's 75–94% versus roughly 2% user-cycle contrast, and a 13-short-module 1.28 replay/build ratio, also used `-M` builds. The panel is not a Mathlib native sample. They motivate questions, not current runtime conclusions. Affected costs/projections remain archived, never scaled by Grid's ratio.

2.4 Outcomes, observation and trust

Process wall, user/system CPU, queue/capture lifecycle and extra rechecks keep their own denominators. Unprofiled, sampled and traced executions are separate con-

ditions. Ordinary error, timeout, cgroup OOM, watchdog interruption, platform loss and never-started dependency are distinct; none proves falsehood. Raw, failed and partial captures remain retained on the PVC.

Import-relative stock `leanchecker` trusts imported declarations. Its `--fresh` mode replays the loaded constant map into an empty kernel. Both use the upstream kernel and designated unsafe/partial exclusions [15]; neither is an independent implementation or an axiom-closure audit. A cheaper trust scope cannot substitute for equivalence.

3 Kernel-evaluation forms are concentrated; runtime weights remain open

RQ1: checking versus recomputing. Table 1 gives the complete, common-boundary source comparison. Unmasked Mathlib text contains eight sites in five files. One is a documentation comment in `Mathlib/Tactic/Echelon/Cert.lean`. Removing comments gives seven in four files; the table uses this common masked boundary, not eight unmasked sites against masked generated code.

The density ratio is 10,668.3, approximately 1.1×10^4. Only 3.5856% of generated-library files contain the form. These are concrete candidates for inspecting recomputation, not an exhaustive runtime classification: expensive reduction can occur without the token, and a positive file can mix computation and symbolic reasoning. The 29 completed no-`-M` random targets sum to 367.024 seconds of native wall intervals and 602.628 user-plus-system CPU seconds. This is not a full-build makespan or population expansion. The separate purposeful Grid and Amplitudes points differ in output/heartbeat scope and do not fill missing strata. Unprofiled process costs do not supply a kernel-cycle partition. [**RESULT: Representative no-`-M` mathematical/computational profiles, regime-specific CPU shares and whole-library cost weights.**] The 3.5856% source-file concentration must not become a cost fraction.

Checking an unchanged computation-based certificate repeats the reductions needed by its justification. It need not repeat the producer's search, but cannot treat an untrusted numerical answer as a theorem. For a mathematical witness a low checking share is possible when source production dominates; it is not established for every mathematical module.

System-design implication. Choose the acceptance boundary before the accelerator. Mathematical witnesses motivate production/checking separation; computational certificates motivate independent kernel work. Tokens and old guarded profiles establish neither a global partition nor an accelerator ceiling.

4 Cheap checking can be hidden by imports and production coupling

RQ2: why independent verification remains costly. Source compilation imports, elaborates, executes tactics and constructs proofs before checking. A post-compilation checker can omit that production, but a new process still locates objects and prepares its environment. A potentially small kernel cost therefore need not imply cheap independent verification.

The earlier no-`-M` stock-checker point completes 15 target replays. For 13 selected short modules its process-wall median is 3.510 seconds, including lookup, imports, replay and cleanup. It is not an exclusive kernel timer or a contemporaneous construction pair. It cannot establish a current construction/replay ratio or its cause by division against an earlier, differently configured build. [**RESULT: Matched no-`-M` build, import-only, checking and cleanup costs; independent-process ratio.**] Repeated imports and coupling are explanations to test, not a residual assigned to an unmeasured stage.

Trust scope changes the work too. Separate no-`-M` controls take 2.195 seconds import-relative and 73.134 seconds fresh-scope for Mathlib Nat.Basic, a 33.32 ratio. A NearPinDeletion fresh-scope control reaches its 180-second wall bound. These are scope contrasts, not interchangeable verification methods. A stream that checks dependencies needs a full-closure baseline.

Prototype: streamed original-kernel replay in one process. O's implementation reads module-data parts, traverses imports, retains their mapped regions and calls the upstream `Kernel.Environment.replay`. A visited set avoids processing an already visited module within the stream. Checking compiled declarations is separate from source elaboration. The implementation also has an import-relative single-module mode and malformed-proof controls. The first complete matched control covers the dependency closure of Mathlib `Data.Nat.Basic`: 1,330 modules and 132,484 declaration entries. Every module is reconstructed with the stock Lean 4.34.1 compiler, requesting `.olean`, `.ilean` and C output. The streamed checker then consumes these newly emitted objects, rather than the initially distributed cache, from one empty checked kernel environment. All commands omit `-M` and use P0–7 with external cgroup protection.

The checking process takes 65.693 seconds versus 622.379 seconds of serial source compiler work (Table 2). This is a comparison between producing and validating artifacts, not a 9.47-fold acceleration of production

Corpus	Lean files	Source MB	Kernel-decision sites	Containing files	Sites/MB
Mathlib `d13f`	8,529	98.450	7	4	0.0711
`openai/math`	163,097	1785.499	1,354,379	5,848	758.544

Table 1: Nested-comment/string-masked `decide +kernel` sites. Decimal MB uses original source bytes. Lexical counts are not time weights.

Complete `Nat.Basic` closure	Native wall (s)
Serial source compiler work, 1,330 calls	622.379
Streamed checking, one process	65.693
Check / source compiler work	10.56%

Table 2: One no-`-M`, same-artifact Mathlib control. The denominator sums compiler subprocess wall intervals, including repeated imports and artifact emission. It is not a parallel Lake makespan.

or serving. The numerator includes process setup, object reads, replay and retirement; the denominator includes each module's imports, elaboration, tactics, first checking and standard artifact emission. Neither clock includes shared-lock wait or coordinator work. The streamed process consumes 65.509 user and 0.163 system CPU seconds; its `wait4` peak RSS is 754,252 KiB.

Forty-two completed single-module-oracle chunks independently reimport each target's environment before native replay; every chunk exits zero and its requested and accepted sets agree. Together they match all 1,330 streamed acceptances. A separate stock-checker preflight covers 13 modules. Three invalid proof terms—a loose variable, unknown constant and wrong sort—are rejected through both streaming and module-local replay. These checks establish differential evidence for the tested artifacts, not universal transparency or an axiom-closure audit. Upstream replay excludes 2,269 unsafe/partial entries in this closure; axioms keep their ordinary Lean semantics. The stock reader's well-formed-object assumption is retained.

The closure includes compiler and standard libraries, not only mathematical theorems. Its 80 Mathlib-namespace modules account for 0.849 seconds of kernel replay and 49.607 seconds of source compiler work. That component ratio (1.71%) omits shared-base checking, reads and process overhead; this namespace also contains tactics and helpers. It cannot replace the full-closure 10.56% process comparison or establish a mathematical-module population estimate. This one-pass, warm-cache control demonstrates a lower-cost checking boundary at complete dependency scope. It does not isolate the causal shares of import reuse and omitted production. [**RESULT: Matched repeated AI-generated mathematical and computational closures, component attribution, uncertainty and full axiom coverage.**]

System-design implication. Expose the cheap checking boundary if one exists: separate production and checking APIs, reuse compatible contexts and charge the complete lifecycle. Warm imports are established engineering [21, 23]; the characterization must show where they matter and what remains expensive.

5 Recomputation motivates parallel original-kernel checking

RQ3: scale without adding trusted proof rules. A monolithic reduction can leave cores unavailable to one checking invocation. Parallelism requires independently checkable claims and a checked connection to the unchanged original proposition. A source split, numeric evaluator or worker success flag alone is insufficient.

Prototype: generated claims and checked assembly. M freezes a real Grid `PointBox.calc` equality and structurally forms 18 scalar-equality leaves. The prototype groups them for one, two, four or eight workers, constructs decision proofs and submits declarations through the original C++ kernel's `Environment.addDeclCore` API. Checked leaves are emitted as modules; a composition proof uses extensionality lemmas and is checked against the original target. Code checks the target, sorry/metavariable absence and expected axiom boundary. These describe the intended acceptance path, not completed evidence.

The controls distinguish original monolithic $k = 0$, the same decomposition with $k = 1$, and $k = 2, 4, 8$. This separates decomposition from parallel execution. Preparation, repeated reductions, serialization/communication, assembly, checking, memory and retirement belong in end-to-end cost. [**RESULT: Complete no-`-M` repetitions, exact proposition/axiom comparison, corrupt-leaf rejection and charged whole-path speedup.**] One frozen equality is not full Grid or a general extraction algorithm. [**RESULT: Shape coverage, heterogeneous-module transfer and failures.**]

A cluster cannot silently trust remote object files or worker verdicts as new proof assertions. Specify what the acceptance authority checks, dependency identity, transfer and recovery. [**RESULT: Multi-machine protocol, native soundness/differential evidence and complete cluster costs.**] Single-host code and multicore timings are not distributed results.

Earlier certificate proofs establish selected feasibility; seven Int/list original/certificate pairs all accept, but currently matched transfer costs remain unestablished. Changing a certified algorithm and parallelizing a fixed decomposition are distinct. Reflection and proof parallelism are established approaches [6, 8, 5].

System-design implication. Expose work at a kernel-checkable boundary; retain original checking rules and the required axiom closure. Keep generation/scheduling outside the trusted decision. Validate decomposition and parallelism separately, then check the complete assembled theorem at equal CPU/memory resources. Faster leaves or transformed subgoal success alone are insufficient.

6 Verified leaves build, outcome and axiom boundaries open

RQ4: what has actually been established? The publisher's default root reaches 130,834 of 163,097 modules. The other 32,263 contain 451,008 of 1,354,379 masked kernel-decision sites, 33.30%. This is not an omitted CPU fraction or a statement that outside files fail. All 562 PlanarPacking and 466 Transonic modules are outside. A successful default build cannot establish all released-artifact coverage.

Source sites, declarations and axioms differ. In the retained 363-object census, 45,661 decision-shaped auxiliaries have 42,610 globally distinct expression-type keys. Source occurrences can expand into several helpers; imported objects use another boundary. Neither these keys nor Lean's branch-local auxiliary cache establishes safe cross-proof reuse [11, 12]. Zero direct native-evaluation tokens does not establish zero transitive axiom dependence. Vericoding source flags and publisher labels similarly do not establish native acceptance or known axiom closure. [**RESULT: Repository-wide declaration reachability, admissions and axiom closure.**]

Settings and observers change outcomes. Six released EPFL exact-program controls fail at recursion depth128 but pass at256/default512 in reconstructed contexts; producer context is unknown. A historical `-M8192` diagnostic changes seven error flags among 200 profiler/control pairs. That is neither full-source accepted-set loss nor a current no-`--M` observer-cost estimate. A C-timing control preserves diagnostics but changes allocation-heartbeat deltas; equal error flags do not establish exact budget accounting. [**RESULT: No-`-M` repeated observer/outcome stability and software coverage.**]

The retained candidate contrasts have different failure/cost boundaries. One miniF2F point spends73.21% of summed physical worker-attempt time on budget-source attempts, not that share of concurrent makespan. ProofNet pass1 has an11.57% budget-source request-time share. AutoVerus's slowest1.6% consumes about3.4%, a slow-tail rather than budget-failure metric. These historical resource points aren't a regime classifier for the library. Ordinary errors and resource exhaustion do not establish falsehood.

System-design implication. Report build domain, checked constants, trusted imports, axiom/admission policy, executable/options, resource accounting and observer configuration. A Boolean result hides these boundaries. Faster delivery of a narrower checking scope isn't equivalent verification.

7 Limits and unfinished evidence

The complete lexical comparison establishes concentration, not runtime prevalence or a universal two-way partition. The no-`--M` sample remains incomplete and purposeful modules receive no population weights. [**RESULT: Current per-regime profiles, expansion estimates and uncertainty.**] Generated-software contexts/provenance differ; software-stage costs remain [**RESULT: pending**].

Streaming and chunking are design-implication prototypes, not accepted-set, soundness, performance or cluster results. Their complete differentials, axiom closure, failures and matching lifecycle costs remain open. Ordinary warm-state reuse and frozen arithmetic oracles are prior mechanisms, not the contribution. The historical GPU screen was slower and removed no timeout; that does not rule out all devices or workloads. All failed, partial and withdrawn evidence remains, including interrupted state-reuse records whose missing entries aren't zero cache opportunity.

8 Related work

Proof-assistant runtime and build performance. Performance engineering of proof assistants predates AI-generated proofs. Lean's architecture and reference-counting work study efficient execution of functional metaprograms [3, 29]. Tebbi and Gross's Ltac profiler attributes nested tactic invocations in Coq [28]. Huch and Wenzel study distributed Isabelle/AFP builds and schedules for multithreaded sessions [5]. These establish runtime, tactic and dependency-scheduling costs as important problems. Our work measures which of these boundaries dominate particular generated artifacts; low realized core use does not by itself establish a new scheduling or runtime technique.

Mathlib compilation and continuous integration. Community profiling has already examined per-file build

times and typeclass, tactic, import and checking phases in Mathlib [27]. We do not claim the first library cost profile. Mathlib's official cache downloads compiled artifacts keyed by toolchain, source and imported dependencies; CI constructs and publishes those artifacts [18]. This makes incremental CI, cached downstream builds and fresh source compilation different cost boundaries. Prior Isabelle-build work also discusses Mathlib's cached workflow in its contemporary setting [5]. We do not import its CI duration or community profiler percentages into our hardware/version denominators. Our new-source study distinguishes default-root coverage, successful artifact growth, CPU/system time and uncompleted probability draws rather than treating a cache hit or a build command's completion as all-source coverage.

Native replay and external proof checkers. The archived `lean4checker` is now distributed with Lean as `leanchecker`. It replays declarations through the official kernel: it is a separate checking pass, not a separately implemented kernel [15]. Carneiro's Lean4Lean reimplements the checker in Lean and evaluates post-compilation replay on major libraries [2]. Lean Kernel Arena evaluates multiple implementations on exported environments, including acceptance, rejection and corner cases [14]. Its instruction-derived virtual CPU scores and a local compiler wall clock have different meanings. These studies address checking performance and trust at post-export or compiled-environment boundaries. Our measurements additionally charge source elaboration, imports, emitted artifacts and failed candidate work; a faster external kernel is useful only for the share at its boundary. Axioms, admissions and native evaluation remain separate trust questions.

RL verification services and prepared proof state. Kimina Lean Server uses concurrent persistent REPLs and cached import headers for RL and proof feedback [21, 23]. Its published throughput study establishes import reuse as a practical service technique, not a new mechanism here. Snapshots retain live proof state for tactic search [25]; stock incremental elaboration reuses prior source state [10]. APE-Bench evaluates file/repository-level proof engineering with pinned library versions and shared infrastructure [31]. Such services and benchmarks establish why reusable contexts matter. Our comparison asks where resource-bounded time and memory go across research mathematics, software artifacts and candidate streams, and whether profiling or changed limits alter their measured verdicts. We do not transfer service speedups to a serial library build.

Reuse, scheduling and certified computation. Fine-grained verification caching [16], tactic/prefix reuse [1, 26], simplifier memoization [9], dynamic dependencies [22, 4] and age-based scheduling [17, 24] are established alternatives. The streaming and chunking prototypes test different regime-specific implications; external search with native checking is prior art.

The gap addressed here is a boundary-consistent, two-regime characterization of released AI-generated formal artifacts, with a research-level library as the primary population and rejected/resource-limited attempts retained as contrasts. It joins source/build coverage to explicitly scoped native costs, treats mapped resident pages and mutable state as distinct metrics, and tests whether observation preserves outcomes. It complements build engineering and checker benchmarks rather than claiming that their bottlenecks are new. Complete software and causal-profile findings remain explicit open results.

9 Conclusion

AI-generated mathematics exposes two verification boundaries: checking mathematical witnesses and recomputing proof-embedded calculations. The matched lexical census shows a large increase concentrated in a small source subset; its full-library time weight remains unknown. Decoupled streaming and original-kernel chunking test different systems responses. **[RESULT: Final no-`-M` regime weights, matched independent-checking costs and whole-path parallel/cluster results.]** Build coverage, reproducibility and axiom closure must accompany every performance or verified claim.

References

[1] Lasse Blaauwbroek, Josef Urban, and Herman Geuvers. The tactician (extended version): A seamless, interactive tactic learner and prover for coq, 2020. Extended version of the CICM paper.

[2] Mario Carneiro. Lean4lean: Verifying a typechecker for lean, in lean, 2025. Version 3, revised 14 September 2025; kernel replay, not source-compilation timing.

[3] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[4] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. Adapton: Composable, demand-driven incremental computation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 156–166, 2014.

[5] Fabian Huch and Makarius Wenzel. Distributed parallel build for the isabelle archive of formal proofs. In *15th International Conference on Interactive*

Theorem Proving (ITP 2024), volume 309 of *LIPIcs*, pages 22:1–22:19, 2024.

[6] Lean community. Mathlib, 2026. Accessed 2026-10-07; year denotes access year.

[7] Lean community. Mathlib4 reference source at d13f23b7. https://github.com/leanprover-community/mathlib4/tree/d13f23b723b8a846827a245b89c10fc7d3f11612/Mathlib, 2026. Pinned Mathlib subtree; local Git revision and unchanged source checked 1 November 2026. Lexical census excludes comments and strings.

[8] Lean contributors. Lean 4.9.0-rc1 ordinary decision tactic implementation. Pinned evalDecide handler; accessed 30 October 2026.

[9] Lean contributors. Lean4.22.0 release: simplifier cache reuse, 2025. Simp section; PR 8880.

[10] Lean contributors. Lean 4.32.0 release: incremental snapshots, 2026. Accessed 2026-10-07; year denotes access year.

[11] Lean contributors. Lean 4.34.1 auxiliary-lemma implementation, 2026. Branch-local cache lookup; accessed 30 October 2026.

[12] Lean contributors. Lean 4.34.1 decision-tactic implementation, 2026. doKernel invokes mkAuxLemma; accessed 30 October 2026.

[13] Lean contributors. Lean 4.34.1 runtime memory-budget checks, 2026. Source and matching SDK disassembly inspected 31 October 2026.

[14] Lean contributors. Lean kernel arena, 2026. Accessed 2026-10-07; year denotes access year.

[15] Lean contributors. lean4checker and the built-in leanchecker, 2026. Official archived README; integrated into Lean from v4.28.0; accessed 31 October 2026.

[16] K. Rustan M. Leino and Valentin Wüstholz. Fine-grained caching of verification results. In *Computer Aided Verification (CAV), Part I*, pages 380–397, 2015.

[17] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal speedup of Las Vegas algorithms. *Information Processing Letters*, 47(4):173–180, 1993.

[18] Mathlib contributors. Mathlib build cache and ci uploads, 2026. Official cache README and Cache/CI.md, accessed 31 October 2026; general CI context, not measured CI cost.

[19] Meta Platforms. Lean Kernel Verify: Compiler-based safety checker for gpu kernels, 2026. Pinned release 129e65bfd8efbf6c45710732d205c941ef65928c; accessed 30 October 2026.

[20] OpenAI math contributors. math: Mathematical manuscripts and supporting proof artifacts, 2026. Pinned source release 4eaee87548250b2682affda7d05d947c05554bae; accessed 30 October 2026.

[21] Project Numina. Kimina lean server, 2026. Accessed 2026-10-07; year denotes access year.

[22] Salsa contributors. The red-green algorithm, 2026. Official algorithm documentation; accessed 30 October 2026.

[23] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[24] Ziv Scully, Mor Harchol-Balter, and Alan Scheller-Wolf. SOAP: One clean analysis of all age-based scheduling policies. arXiv:1712.00790v2, 2018.

[25] Austin Shen and Yunong Shi. Keep the proof state live: Snapshotting for efficient tactic search in lean 4, 2026.

[26] SHRED contributors. SHRED: Reuse repeated work across thousands of Lean proof attempts, 2026. Software and experimental artifact; revision a6616e99534ce777dcebfd56566eaaffb63997a8; accessed 16 October 2026.

[27] Michael Stoll. Ways to speed up mathlib. Lean community Zulip archive, 25 April 2024, 2024. Community measurement report; no timing or phase percentage transferred to our experiments.

[28] Tobias Tebbi and Jason Gross. A profiler for ltac. CoqPL Workshop abstract, 2015.

[29] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: Reference counting optimized for purely functional programming. In *Proceedings of the International Symposium on Implementation and Application of Functional Languages (IFL '19)*, 2020.

[30] Vericoding contributors. Vericoding: released model-tagged programs and final experiment tables. Artifact repository, revision b265a3b2bce31e09d264306f419e810d02f98f44, 2026. Inspected 31 October 2026; final outputs do not constitute complete verification trajectories.

[31] Huajian Xin, Luming Li, Xiaoran Jin, Jacques Fleuriot, and Wenda Li. Ape-bench: Evaluating automated proof engineering for formal math libraries, 2026. Version 3, revised 2 February 2026.

[32] Zhe Ye, Hantao Lou, Yuechun Sun, et al. Vero: Can ai agents build formally verified software repositories?, 2026. Official repository inspected at 0a7325df9e9e6dbc275c0ad483b3d1cbe38d9b09; evaluated run outputs not established.